

Convolutional Neural Networks

A **Convolutional Neural Network (CNN)** is a specialised type of deep learning network designed to **process and analyse visual data**, such as images and videos. Unlike fully connected networks, CNNs automatically extract hierarchical features from data, making them highly effective for tasks like image classification, object detection, and face recognition.

Model Type	Key Challenges	Explanation
Traditional ML	High Dimensionality	Images have thousands to millions of pixels, making computation slow and expensive.
	Manual Feature Engineering	Cannot automatically extract features like edges, shapes, or textures; requires handcrafted features.
	Ignoring Spatial Relationships	Treats all pixels independently, losing important spatial information.
Multilayer Perceptron (MLP)	Scalability Issues	Fully connected layers create too many parameters for large images.
	Poor Spatial Understanding	Does not capture local pixel patterns or spatial structure.
	Overfitting Risk	High number of parameters can lead to memorising training images instead of learning general features.
Artificial Neural Network (ANN)	Limited Feature Extraction	Cannot automatically learn hierarchical features from raw images.
	Difficulty with Complex Patterns	Struggles with non-linear and high-dimensional data.
	Inefficient for Large Images	Processing high-resolution images requires too many weights, making training impractical.

Visual Cortex and Digital Image

CNNs are inspired by the **human visual system**, which processes visual information hierarchically, detecting simple patterns first and then combining them into complex shapes. This biological insight guides how CNNs analyse digital images effectively.

- **Visual Cortex Inspiration:**
 - Detects **edges, lines, and basic colors** in the early stages of vision.
 - Higher layers integrate these simple features into **complex patterns and objects**, mimicking hierarchical processing.
- **Digital Representation of Images:**
 - Images are stored as **grids of pixels**, where each pixel encodes the intensity of light.
 - **RGB Images:** Each pixel has three values—**Red, Green, Blue**—ranging from 0 to 255.
 - The spatial arrangement of pixels corresponds to the visual structure, allowing CNNs to detect local patterns like edges and textures.
- The combination of hierarchical feature detection in the visual cortex and the pixel-based structure of digital images is what enables CNNs to **learn spatial patterns and progressively complex features** automatically.

Core Components

Component	Explanation
Kernel / Filter	A small matrix that slides over the input image to detect features like edges, textures, or color gradients.
Feature Map	The output of a convolution operation after applying a filter; highlights where specific features appear in the image.
Stride	The number of pixels the filter moves at each step. Larger stride → smaller output feature map.
Padding	Adding extra pixels (usually zeros) around the image to preserve spatial dimensions or control output size.

Pooling	Downsampling operation to reduce feature map dimensions while retaining important information. Common types: Max Pooling and Average Pooling.
----------------	---

Popular CNN Architectures

1. LeNet

- Developed in the 1990s by Yann LeCun, LeNet was one of the **earliest successful CNNs**, designed for **handwritten digit recognition** (like MNIST dataset).
- Architecture is **small and simple**, consisting of **convolutional layers followed by pooling and fully connected layers**.
- Demonstrated the power of CNNs in **automatically learning features** from images, reducing the need for manual feature engineering.

2. VGGNet

- Developed by the Visual Geometry Group (VGG) at Oxford, this network is **much deeper than LeNet**.
- Uses **uniform 3×3 convolution filters stacked together**, which allows the network to learn **fine-grained features** hierarchically.
- Highly effective for **large-scale image classification tasks**, such as ImageNet.
- The uniform structure makes it **easy to implement and scale**, but deeper layers result in more parameters and higher computational cost.

3. ResNet

- Introduces **residual connections (skip connections)** that allow the network to **bypass certain layers**.
- Solves the **vanishing gradient problem**, which often occurs in very deep networks where gradients become too small for effective learning.
- Enables training of **extremely deep CNNs** (hundreds of layers) while maintaining stable performance.

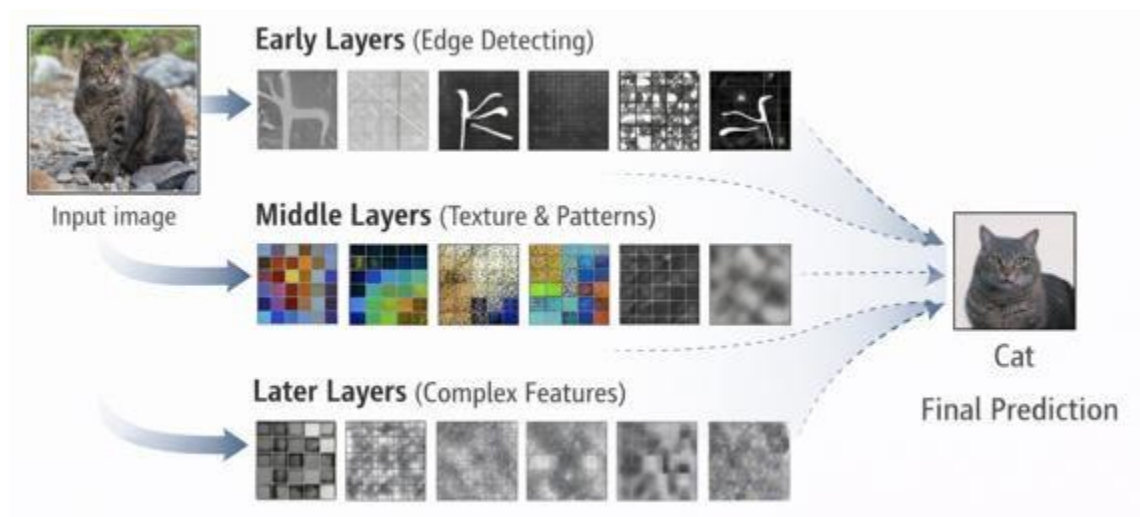
- Widely used in modern computer vision tasks, including object detection and segmentation.

CNN Visualiser

It is a tool that helps you **see what a Convolutional Neural Network is learning** at each layer.

- As an image passes through a CNN, each **layer extracts different features**:
 - **Early layers**: detect simple patterns like edges, lines, and colors.
 - **Middle layers**: detect more complex shapes and textures.
 - **Later layers**: detect high-level objects or patterns in the image.
- The visualiser can **display the feature maps**, which are the outputs of each convolutional layer, so you can **observe which parts of the image activate the network**.

Following image illustrates the CNN visualiser process



CNN Workflow

1. Data Loading and Preprocessing



Every CNN model begins with data preparation to ensure the images are in a suitable format for learning.

- **Image Representation:** Images are stored as numerical grids in the format **Height × Width × Channels**, where grayscale images contain **one channel** representing intensity values, RGB images contain **three channels** representing red, green, and blue components, and each pixel typically has intensity values ranging from **0 to 255**.
- **Preprocessing Steps**
 - **Resizing:** All images are resized to the same dimensions so that the CNN can process them consistently in batches.
 - **Normalisation:** Pixel values are scaled (commonly between 0–1 or -1 to 1) to ensure stable and faster training by preventing large numerical variations.
 - **Train–Validation–Test Split:** The dataset is divided into three parts, where the **training set** is used to learn patterns, the **validation set** is used to tune hyperparameters and monitor overfitting, and the **test set** is used to evaluate the final model performance on unseen data.

2. Fundamental Components of CNN

CNNs are built using specialised layers that work together to extract features and make predictions from images.

- **Convolution Layer:** The convolution layer applies small filters (kernels) that slide across the image to detect local patterns such as edges, textures, and shapes by performing element-wise multiplication and summation.
- **Activation Function:** An activation function introduces non-linearity into the network, allowing it to learn complex patterns, with **ReLU** being the most commonly used due to its efficiency and ability to reduce vanishing gradient issues.
- **Pooling Layer:** The pooling layer reduces the spatial dimensions of feature maps while preserving important information, which helps decrease computation and reduce overfitting.
- **Flatten Layer:** The flatten layer converts the 2D feature maps into a 1D vector so that the data can be passed into fully connected layers.



- **Fully Connected Layer:** The fully connected layer combines all extracted features and performs the final classification or regression task.

3. Forward Propagation in CNN

Forward propagation is the process in which the input image passes through all layers of the network to generate predictions.

- The image first passes through convolution and activation layers to extract features.
- Pooling layers reduce dimensions while keeping key information.
- The output is flattened and passed through fully connected layers.
- The final layer produces predictions based on the learned features.

4. Output Layer

The structure of the output layer depends on whether the task is classification or regression.

- **For Binary Classification:** A sigmoid activation function is used to output a probability value between 0 and 1, and Binary Cross-Entropy is commonly used as the loss function.
- **For Multi-Class Classification:** A softmax activation function converts outputs into class probabilities, and Categorical Cross-Entropy is typically used as the loss function.
- **For Regression:** A linear activation function is used to predict continuous values, and Mean Squared Error (MSE) or Mean Absolute Error (MAE) is used to measure prediction error.

5. Backpropagation and Weight Update

Backpropagation is the learning mechanism that updates the network's weights to reduce prediction error.



- The loss function measures the difference between predicted and actual values.
- Gradients are calculated using the chain rule to determine how much each weight contributed to the error.
- An optimiser updates the weights in the opposite direction of the gradient to minimise loss.
- This process repeats for multiple epochs until the model improves.

6. Optimisers

Optimisers control how the model updates its weights during training.

- **Adam:** Adam adapts the learning rate for each parameter individually, allowing faster and more stable convergence in most practical cases.
- **SGD (Stochastic Gradient Descent):** SGD updates weights step-by-step using small batches of data and can sometimes provide better generalisation.
- **RMSProp:** RMSProp adjusts learning rates based on recent gradients and performs well in complex deep learning tasks.

7. Regularisation Techniques

Regularisation techniques help prevent overfitting and improve generalisation.

- **Dropout:** Dropout randomly disables a fraction of neurons during training, forcing the network to learn more robust features.
- **L1 / L2 Regularisation:** L1 and L2 regularisation add penalty terms to the loss function to prevent the model from learning excessively large weights.
- **Data Augmentation:** Data augmentation increases dataset diversity by applying transformations like rotation, flipping, and zooming to training images.



- **Early Stopping:** Early stopping monitors the validation loss during training and automatically stops the training process when the validation performance stops improving, preventing the model from overfitting the training data.

8. Hyperparameter Tuning

Hyperparameters control the learning behavior of the CNN and must be carefully selected.

- **Learning Rate:** The learning rate determines how large the weight updates are, and choosing an appropriate value is crucial for stable training.
- **Number of Layers:** The number of layers determines the depth of feature extraction, where shallow networks suit simple tasks and deeper networks handle more complex patterns.
- **Number of Filters:** Increasing filters allows the model to learn more feature types but also increases computational cost.
- **Batch Size and Epochs:** Batch size controls how many samples are processed at once, and epochs determine how many times the entire dataset is used for training.

Recurrent Neural Networks

Sequential Data and Its Importance

Sequential data is data in which the **order of elements matters**, and each element may depend on previous elements in the sequence. Examples include sentences in text, speech signals, time-series data, and stock price movements.

- The meaning or value at time t depends on time $t-1$.
- Removing or changing the order changes the overall meaning.
- Context plays a critical role in understanding the data.

To properly understand sequential data, a model must:



- **Remember previous inputs** so that earlier information influences later predictions.
- **Capture temporal dependencies**, meaning it understands how current values depend on past values.
- **Maintain context across time steps** instead of treating each input independently.

RNN addresses these challenges by:

- **Introducing a hidden state (memory)** that stores information from previous steps.
- **Passing this hidden state from one time step to the next**, allowing information to flow through the sequence.
- **Reusing the same weights at every time step**, which helps the network learn patterns across time without increasing model complexity.

Background of RNN

Perceptron: A Perceptron is the simplest type of neural network with a single layer that performs linear classification. It processes inputs independently and cannot handle complex patterns or sequential dependencies.

- Processes each input independently with no memory of previous inputs.
- Cannot capture relationships between past and present data points.
- Ignores the order of data in a sequence.
- Works only for simple, linearly separable problems.
- Not suitable for time-series, text, or speech data.

MLP (Multi-Layer Perceptron): An MLP is a feedforward neural network with multiple hidden layers that can learn non-linear relationships.

However, it treats each input as independent and does not maintain memory of previous inputs.

- Treats inputs as fixed-size independent vectors.
- Does not maintain contextual information across time steps.



- Cannot naturally capture temporal dependencies.
- Requires manual feature engineering to represent sequence order.
- Struggles with variable-length sequence inputs.

RNN (Recurrent Neural Network): An RNN is designed to process sequential data by introducing a hidden state that carries information across time steps.

This internal memory enables it to capture temporal dependencies and contextual relationships in sequences.

- Maintains a hidden state that stores past information.
- Captures temporal dependencies through recurrent connections.
- Processes data sequentially, preserving order.
- Reuses weights across time steps for efficient learning.
- Effectively handles time-series, language, and other sequential tasks.

Recurrent Neural Network (RNN) Architecture

A **Recurrent Neural Network (RNN)** is designed to handle **sequential data** such as time series, text, speech, or video. Unlike traditional feedforward networks, RNNs maintain a **memory of previous inputs** using hidden states.

- **Basic RNN Structure (Single Layer)**

A Recurrent Neural Network (RNN) is designed to process **sequential data** such as text, speech, or time-series signals. Unlike traditional neural networks, it remembers previous inputs using a hidden state.

An RNN consists of:

- **Input Layer** → Sequential input x_t
- **Hidden Layer** → Stores memory h_t
- **Output Layer** → Produces prediction y_t

At each time step, the RNN updates its memory using the current input and the previous hidden state:

$$\begin{aligned}h_t &= f(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \\ y_t &= W_{hy}h_t + b_y\end{aligned}$$

Where:

- x_t = Input at time step t
- h_t = Hidden state (memory of the network)
- h_{t-1} = Previous hidden state
- W = Weight matrices (learned during training)
- f = Activation function (Tanh / ReLU)

This recurrent connection allows the model to capture dependencies across time.

- **Unrolled RNN Architecture:** RNN unfolding or unrolling is the process of expanding the recurrent structure over time steps. During unfolding each step of the sequence is represented as a separate layer in a series illustrating how information flows across each time step.

This unrolling enables backpropagation through time (BPTT) a learning process where errors are propagated across time steps to adjust the network's weights enhancing the RNN's ability to learn dependencies within sequential data.

$x_1 \rightarrow [\text{RNN Cell}] \rightarrow h_1 \rightarrow y_1$

$x_2 \rightarrow [\text{RNN Cell}] \rightarrow h_2 \rightarrow y_2$

$x_3 \rightarrow [\text{RNN Cell}] \rightarrow h_3 \rightarrow y_3$

Each cell:

- Shares the same weights
- Passes hidden state to next time step
- Learns temporal dependencies

Even though it looks like multiple layers, it is actually the **same cell repeated over time**.



- **Types of RNN Architectures:** Depending on input and output structure, RNNs can be designed differently.

a) Many-to-One: Multiple inputs produce a single output. Used when the entire sequence determines one final prediction.

Used in:

- Sentiment Analysis
- Text Classification

Example: $x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4 \rightarrow y$ (Only final output is used)

b) One-to-Many: Single input produces multiple outputs over time.

Used in: Image Captioning

Example: $x \rightarrow y_1 \rightarrow y_2 \rightarrow y_3 \rightarrow y_4$

c) Many-to-Many (Same Length): Each input time step produces a corresponding output.

Used in:

- POS Tagging
- Sequence labeling

d) Many-to-Many (Different Length): Input and output sequence lengths are different.

Used in: Machine Translation

This typically uses an **Encoder-Decoder architecture**, where one RNN encodes the input and another decodes it into output.

- **Advanced RNN Variants**



Basic RNN suffers from the **vanishing gradient problem**, which makes learning long-term dependencies difficult.

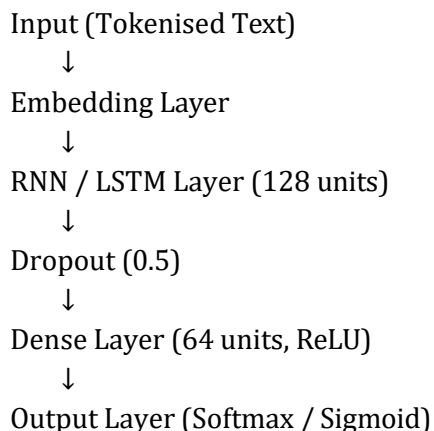
To solve this, advanced architectures are used:

- **Long Short-Term Memory (LSTM):** Introduce a memory mechanism to overcome the vanishing gradient problem. Each LSTM cell has three gates:
 - **Input Gate:** Controls how much new information should be added to the cell state.
 - **Forget Gate:** Decides what past information should be discarded.
 - **Output Gate:** Regulates what information should be output at the current step. This selective memory enables LSTMs to handle long-term dependencies, making them ideal for tasks where earlier context is critical.
- **Gated Recurrent Units (GRUs):** Simplify LSTMs by combining the input and forget gates into a single update gate and streamlining the output mechanism. This design is computationally efficient, often performing similarly to LSTMs and is useful in tasks where simplicity and faster training are beneficial.

These models include **gating mechanisms** that control what information to remember and what to forget.

- **Architecture for Text Classification**

Task: Sentiment Analysis: This architecture processes text step-by-step and predicts whether sentiment is positive or negative.





Here:

- Embedding converts words into vectors
- RNN/LSTM learns context
- Dropout prevents overfitting
- Dense layer refines features
- Output layer predicts final class

Trainable Parameters in RNN

Trainable parameters in an RNN are the **weights and biases** that the model learns during training to minimise error. These parameters are updated using backpropagation through time (BPTT).

Input-to-Hidden Weights (W_x)

- Connect the **current input** (x_t) to the hidden state.
- Determine how much influence the current input has on memory update.
- Shape: $(hidden_size \times input_size)$

Hidden-to-Hidden Weights (W_h)

- Connect the **previous hidden state** (h_{t-1}) to the current hidden state.
- Responsible for carrying past information forward.
- Shape: $(hidden_size \times hidden_size)$
- This is the key parameter that enables the “recurrent” behavior.



Hidden-to-Output Weights (W_y)

- Connect the hidden state to the output layer.
- Control how internal memory translates into predictions.
- Shape: $(output_size \times hidden_size)$

Bias Terms (b_h and b_y)

- b_h : Bias for hidden layer.
- b_y : Bias for output layer.
- Help shift activation functions for better learning flexibility.

RNN Workflow

1. Data Loading and Preprocessing: Every RNN model begins with preparing sequential data in a structured format.

► **Sequence Representation:** Data is arranged in **time steps**, typically in the format:

- **(Batch Size $\times$ Sequence Length $\times$ Features)**
- Examples:
 - Text $\rightarrow$ sequence of words/tokens
 - Time-series $\rightarrow$ sequence of numerical values
 - Speech $\rightarrow$ sequence of audio frames

► **Preprocessing Steps**

- **Tokenisation (for text data):** Convert sentences into tokens (words/characters).
- **Encoding:** Convert tokens into numerical format using techniques like integer encoding or embeddings.



- **Padding/Truncation:** Ensure all sequences have equal length by adding padding tokens or trimming long sequences.
- **Normalisation (for numerical sequences):** Scale values (0-1 or -1 to 1) for stable training.
- **Train-Validation-Test Split:**
 - Training set → Learn sequence patterns
 - Validation set → Tune hyperparameters & monitor overfitting
 - Test set → Evaluate final model performance

2. Fundamental Components of RNN: RNNs are built using recurrent units that process data step-by-step.

- **Input Layer:** Accepts sequence data at each time step (x_t).
- **Recurrent (Hidden) Layer:** Maintains a hidden state (h_t) that stores past information.
It combines current input (x_t) and previous hidden state (h_{t-1}).
- **Activation Function:** Commonly **tanh** or **ReLU** to introduce non-linearity.
- **Output Layer:** Produces predictions either at each time step or only at the final time step.

3. Forward Propagation in RNN: Forward propagation happens sequentially across time steps.

- At time step t :
 - Input x_t is received.
 - Combined with previous hidden state h_{t-1} .
 - New hidden state h_t is computed.
 - Output y_t is generated (if required).



- This process repeats until the end of the sequence. Unlike CNNs, RNN processes data **step-by-step instead of spatially**.

4. **Output Layer:** The output structure depends on the task:

- **Sequence-to-One (Many-to-One)**

Example: Sentiment analysis

- Final hidden state is used.
- Sigmoid → Binary classification
- Softmax → Multi-class classification

- **Sequence-to-Sequence (Many-to-Many)**

Example: Language translation

- Output generated at every time step.

- **Regression Tasks**

- Linear activation
- Loss: MSE or MAE

5. **Backpropagation Through Time (BPTT):** Since RNNs handle sequential data, **BPTT** is employed to update the network's parameters. The loss function $L(\theta)$ depends on the final hidden state h_3 , and each hidden state is influenced by its preceding states, forming a chain of sequential dependencies:

h_3 depends on h_2 , h_2 depends on h_1 , and h_1 depends on h_0 .

In BPTT, gradients are propagated backward across all time steps. This allows the network to adjust its parameters based on temporal dependencies in the sequence.

1. **Simplified Gradient Computation:**

$$\frac{\partial L(\theta)}{\partial W} = \frac{\partial L(\theta)}{\partial h_3} \cdot \frac{\partial h_3}{\partial W}$$

2. **Accounting for Sequential Dependencies:**

Each hidden state is updated based on the previous state:

$$h_3 = \sigma(W \cdot h_2 + b)$$

Gradients are computed for every state, taking into account how earlier states influence later ones.

3. **Explicit and Implicit Gradient Contributions:**

The gradient can be decomposed into direct (explicit) and indirect (implicit) contributions from all preceding hidden states:

$$\frac{\partial h_3}{\partial W} = \frac{\partial h_3}{\partial W} + \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial W} + \dots$$

4. **Final Gradient Formula:**

The overall derivative of the loss function with respect to the weight matrix W sums contributions from all time steps:

$$\frac{\partial L(\theta)}{\partial W} = \frac{\partial L(\theta)}{\partial h_3} \cdot \sum_{k=1}^3 \frac{\partial h_3}{\partial h_k} \cdot \frac{\partial h_k}{\partial W}$$

This step-by-step backward computation across time is the core mechanism behind **Backpropagation Through Time**, enabling RNNs to learn from sequential dependencies effectively.

6. **Optimisers:** Optimisers update weights to minimise loss.

- **Adam:** Adaptive learning rates; widely used for faster convergence.
- **SGD:** Updates weights using mini-batches; good generalisation.
- **RMSProp:** Effective for sequential models and recurrent training.

7. **Regularisation Techniques:** To prevent overfitting and improve generalisation:

- **Dropout:** Applied to non-recurrent connections to reduce overfitting.
- **L1 / L2 Regularisation:** Penalises large weights.
- **Early Stopping:** Stops training when validation loss stops improving.



- **Gradient Clipping:** Prevents exploding gradients in long sequences.

8. Hyperparameter Tuning

- **Learning Rate:** Controls weight update size.
- **Hidden Units:** Determines memory capacity.
- **Number of Layers:** Stacked RNN layers for deeper learning.
- **Sequence Length:** Number of time steps processed at once.
- **Batch Size and Epochs:** Control training cycles.